AfriOS

Guide de build complet

Du clone au build OS complet

Procédure pas-à-pas pour reproduire le build AfriOS sur une machine outillée : pré-requis, clone, vendorisation, build noyau (4 archs), build firmware UEFI, build 5 couches de compatibilité.

VERSION
1.0

DATE
03 September 2026

AUTEUR
Équipe AfriOS
Document technique AfriOS — github.com/ErdisKodjo/System

STATUT
Public

Table des matières

Table des matières 2
1. Introduction 5
1.1 Public cible . . . 5
1.2 Conventions . . . 5
1.3 Durée totale estimée . . . 5
1.4 Pré-requis matériels . . . 5
2. Pré-requis système 7
2.1 Outils de build de base . . . 7
2.2 Cross-compilers pour les 4 architectures . . . 7
2.3 Compilateurs spécialisés . . . 7
2.4 Émulateur QEMU (pour tests runtime) . . . 7
2.5 Vérification des pré-requis . . . 7
3. Clone du dépôt 9
3.1 Clone standard . . . 9
3.2 Structure du dépôt . . . 9
3.3 Lecture du README . . . 9
4. Vendorisation des dépendances 10
4.1 Vue d'ensemble des dépendances . . . 10
4.2 Vendorisation minimale (build sans firmware) . . . 10
4.3 Vendorisation complète (avec EDK2) . . . 10
4.4 Vérification post-vendorisation . . . 11
4.5 Mise à jour d'une dépendance . . . 11
5. Build des 5 couches de compatibilité 12
5.1 Build one-shot (recommandé) . . . 12
5.2 Build manuel module par module . . . 12
6. Build du noyau (4 architectures) 14
6.1 Vue d'ensemble . . . 14
6.2 Build ARM64 (Raspberry Pi 4, Pine64, Ampere Altra) . . . 14
6.3 Build x86_64 (desktops, serveurs) . . . 14
6.4 Build RISC-V 64 (SiFive, QEMU virt) . . . 14
6.5 Build MCU Cortex-M (STM32, nRF52, RP2040) . . . 14
6.6 Build host-mock (pour tests sans QEMU) . . . 15

7. Build du firmware UEFI 16
7.1 Pré-requis EDK2 . . . 16
7.2 Configuration de l'environnement EDK2 . . . 16
7.3 Build firmware X64 (QEMU, serveurs x86) . . . 16
7.4 Build firmware AARCH64 (Rpi 4, Pine64) . . . 16
7.5 Build pour une carte spécifique . . . 16
7.6 Test firmware sur QEMU . . . 16
8. Tests et validation 18
8.1 Niveau 1 — Tests HAL (sans QEMU) . . . 18
8.2 Niveau 2 — Tests syntaxiques CI . . . 18
8.3 Niveau 3 — Tests de compatibilité . . . 18
8.4 Niveau 4 — Boot test sur QEMU . . . 18
8.5 Script one-shot de validation . . . 18
9. Dépannage 20
9.1 Erreurs CMake . . . 20
9.2 Erreurs cross-compilation . . . 20
9.3 Erreurs EDK2 . . . 20
9.4 Erreurs tests . . . 21
10. Check-list finale du build complet 22
10.1 Pré-requis . . . 22
10.2 Acquisition . . . 22
10.3 Build couches compatibilité . . . 22
10.4 Build noyau . . . 22
10.5 Build firmware . . . 23
10.6 Validation . . . 23
10.7 Artifacts finaux attendus . . . 23
11. Annexe A — Build sur Windows via WSL2 24
11.1 Pourquoi pas de build natif Windows ? . . . 24
11.2 Étape 1 — Installer WSL2 + Ubuntu 24.04 . . . 24
11.3 Étape 2 — Installer les pré-requis (une fois) . . . 24
11.4 Étape 3 — Cloner AfriOS dans WSL . . . 25
11.5 Étape 4 — Vendorisation + build complet . . . 25
11.6 Étape 5 — Accéder aux artifacts depuis Windows . . . 25
11.7 Étape 6 — Test QEMU (avec KVM si WSLg activé) . . . 25
11.8 Optimisations WSL2 . . . 26

11.9 Limites connues de WSL2 pour AfriOS 26
11.10 Alternative : Docker Desktop 26
12. Raccourci one-shot **27**
11.1 Sortie attendue du setup.sh --all 27
13. Conclusion **28**

1. Introduction

Ce document est un guide pas-à-pas pour reproduire un build complet d'AfriOS sur une machine outillée. Il part du postulat que vous venez de cloner le dépôt et que vous souhaitez aboutir à un build complet : firmware UEFI multi-architectures, noyau pour les 4 architectures supportées (ARM64, x86_64, RISC-V, MCU Cortex-M), et les 5 couches de compatibilité applicative (Windows, Android, iOS/macOS, HarmonyOS, DXVK).

Le guide couvre également la vendorisation des dépendances externes (EDK2, Wine, ART, Darling, etc.) qui est un prérequis pour le build firmware. Enfin, il documente le script one-shot setup.sh qui automatise l'essentiel, ainsi que les commandes manuelles pour chaque étape si vous souhaitez comprendre ou contrôler chaque phase.

1.1 Public cible

Ce guide s'adresse à un développeur ou un intégrateur système connaissant Linux, CMake, et les toolchains de cross-compilation. Aucune connaissance préalable d'AfriOS n'est requise — le guide est self-contained et peut être suivi de bout en bout sans se référer à d'autres documents.

1.2 Conventions

Les commandes shell sont préfixées par $ pour indiquer un prompt utilisateur normal (non-root). Les commandes nécessitant root sont préfixées par #. Les blocs de code sans prompt indiquent un fichier à créer ou modifier. Les chemins relatifs sont donnés par rapport à la racine du dépôt cloné.

1.3 Durée totale estimée

Étape	Durée	Téléchargement
1. Pré-requis système	~10 min	~500 MB
2. Clone du dépôt	~30 s	~5 MB
3. Vendorisation dépendances	~30-60 min	~6 GB
4. Build 5 couches compat	~5 min	—
5. Build noyau (4 archs)	~15 min	—
6. Build firmware UEFI	~30 min	—
7. Tests & validation	~5 min	—
Total (build complet)	~2 heures	~6.5 GB

1.4 Pré-requis matériels

Pour le build complet (firmware inclus), comptez : 10 GB d'espace disque libre (dépôt + EDK2 + build artifacts), 4 GB de RAM minimum (8 GB recommandé pour le build EDK2 parallèle), et une connexion Internet stable à ~10 Mbps minimum pour les downloads EDK2 et dépendances. Le build firmware est CPU-intensif ; un processeur 4 cœurs ou plus est recommandé.

2. Pré-requis système

Cette section liste les paquets à installer avant de commencer. Les commandes sont données pour Debian/Ubuntu ; adaptez-les pour votre distribution (Fedora, Arch, etc.).

2.1 Outils de build de base

```
# Debian/Ubuntu sudo apt-get update sudo apt-get install -y \ build-essential gcc g++ make cmake ninja-build \ python3 python3-pip git curl wget unzip \ pkg-config libssl-dev # Version minimale requise # gcc >= 11 (recommandé 13+) # cmake >= 3.16 # python3 >= 3.10 # git >= 2.20
```

2.2 Cross-compilers pour les 4 architectures

AfriOS supporte 4 architectures. Pour builder le noyau pour chacune, vous devez installer le cross-compiler dédié. Le tableau liste les paquets Debian/Ubuntu pour chaque architecture.

Architecture	Paquet Debian/Ubuntu	Commande gcc
ARM64 (AArch64)	gcc-aarch64-linux-gnu	aarch64-linux-gnu-gcc
x86_64	(natif, déjà installé)	gcc
RISC-V 64	gcc-riscv64-linux-gnu	riscv64-linux-gnu-gcc
MCU Cortex-M	gcc-arm-none-eabi	arm-none-eabi-gcc

```
# Installer les 3 cross-compilers (x86_64 = natif) sudo apt-get install -y \ gcc-aarch64-linux-gnu \ gcc-riscv64-linux-gnu \ gcc-arm-none-eabi # Vérification aarch64-linux-gnu-gcc --version riscv64-linux-gnu-gcc --version arm-none-eabi-gcc --version
```

2.3 Compilateurs spécialisés

Pour compiler les frameworks iOS/macOS (UIKit, Foundation, CoreAnimation) en Objective-C, vous avez besoin de gobjc ou clang. Pour compiler les tests Windows en .exe, installez mingw-w64.

```
# Objective-C (pour afros-incompat-engine frameworks .m) sudo apt-get install -y gobjc gobjc-14 \ clang libobjc-10-dev # MinGW-w64 (pour tests Windows .exe) sudo apt-get install -y \ x86_64-w64-mingw32-gcc \ x86_64-w64-mingw32-g++ \ mingw-w64-tools
```

2.4 Émulateur QEMU (pour tests runtime)

QEMU est optionnel pour le build mais requis pour tester les binaires cross-compilés sur votre machine de dev (par exemple, booter le noyau ARM64 sur QEMU virt).

```
sudo apt-get install -y \ qemu-system-arm \ qemu-system-x86 \ qemu-system-riscv32 \ qemu-system-misc \ qemu-user-static
```

2.5 Vérification des pré-requis

Le script scripts/check-deps.sh vérifie que tous les outils sont en place. Lancez-le avant de continuer :

```
$ ./scripts/setup.sh --check-deps Phase: check-deps ✓ gcc 14.2.0 ✓ g++ 14.2.0 ✓ cmake 4.4.3 ✓ python3 3.11 ✓ aarch64-linux-gnu-gcc 14.2.0 ✓ riscv64-linux-gnu-gcc 14.2.0 ✓ arm-none-eabi-gcc 14.2.0 ✓ x86_64-w64-mingw32-gcc 12.0 ✓ qemu-system-aarch64 8.2.0 [ok] all dependencies satisfied
```

3. Clone du dépôt

Le dépôt AfriOS est hébergé sur GitHub à l'adresse github.com/ErdisKodjo/System. Le clone est petit (~5 MB) car les dépendances externes sont vendorisées séparément via fetch-deps.sh.

3.1 Clone standard

```
$ git clone https://github.com/ErdisKodjo/System.git AfriOS $ cd AfriOS # Vérification $ ls -la total 32 drwxr-xr-x 3 user user 4096 Sep 3 14:00 . drwxr-xr-x 12 user user 4096 Sep 3 14:00 .. drwxr-xr-x 8 user user 4096 Sep 3 14:00 .git drwxr-xr-x 2 user user 4096 Sep 3 14:00 .github -rw-r--r-- 1 user user 34 Sep 3 14:00 .gitignore drwxr-xr-x 4 user user 4096 Sep 3 14:00 AfriOS drwxr-xr-x 3 user user 4096 Sep 3 14:00 ci-workflows -rw-r--r-- 1 user user 5617 Sep 3 14:00 README.md drwxr-xr-x 2 user user 4096 Sep 3 14:00 docs drwxr-xr-x 2 user user 4096 Sep 3 14:00 scripts drwxr-xr-x 2 user user 4096 Sep 3 14:00 tests
```

3.2 Structure du dépôt

```
AfriOS/ +-- AfriOS/ Code source principal | +-- FirmwareHybride/ Firmware UEFI/EDK2 | +-- AfriOS/OS/ Système d'exploitation | +-- afros-core/ Noyau + HAL + pilotes | +-- afros-corebridge-core/ Orchestrateur multi-runtime | +-- afros-dxvk/ DirectX -> Vulkan | +-- afros-winbridge/ Compatibilite Windows (Wine) | +-- afros-androsandbox/ Compatibilite Android | +-- afros-incompat-engine/ Compatibilite iOS/macOS (Darling) | +-- afros-harmonygate/ Compatibilite HarmonyOS | +-- afros-network/ Stack reseau intelligent | +-- afros-storage/ Gestion stockage (AfrosFS) | +-- afros-power-management/ Gestion energie | +-- afros-sdk/ SDK publique | +-- cmake/toolchains/ 4 toolchains cross-compilation | +-- apps/demo_app/ App de demo +-- ci-workflows/ GitHub Actions (trackés) +-- docs/ Documentation +-- scripts/ Automatisation (fetch-deps, setup, etc.) +-- tests/ Tests compatibilité 5 plateformes +-- .github/ CodeOWNERS, PR template, ISSUE_TEMPLATE
```

3.3 Lecture du README

Le fichier README.md à la racine contient un Quick Start qui résume les commandes essentielles. Le présent document est plus détaillé et couvre l'ensemble du build OS complet.

4. Vendorisation des dépendances

AfriOS dépend de 9 bibliothèques externes pour le build complet : EDK2 (firmware UEFI), Wine (compat Windows), ART (compat Android), Darling (compat iOS/macOS), HarmonyOS SDK, Vulkan-Headers, glslang, Mesa, libiconv. Ces dépendances ne sont PAS commitées dans le dépôt (trop volumineuses) mais sont vendorisées via un script reproductible.

4.1 Vue d'ensemble des dépendances

Dépendance	Version pin	Taille	Requise pour
EDK2	edk2-stable202408	~5 GB	Firmware UEFI
Wine	wine-9.0	~500 MB	afros-winbridge (optionnel)
ART	android-14.0.0_r1	~300 MB	afros-androsandbox (optionnel)
Darling	0.1.20240301	~200 MB	afros-incompat-engine (optionnel)
HarmonyOS SDK	5.0.0-Release	~150 MB	afros-harmonygate (optionnel)
Vulkan-Headers	v1.3.290	~10 MB	afros-dxvk
glslang	14.2.0	~30 MB	afros-dxvk (HLSL compiler)
Mesa	mesa-24.0.3	~500 MB	Vulkan ICD (optionnel)
libiconv	v1.17	~5 MB	Conversions charset

4.2 Vendorisation minimale (build sans firmware)

Pour un build sans le firmware UEFI (noyau + compat layers uniquement), vous n'avez besoin que de Vulkan-Headers et glslang (pour DXVK). Wine, ART, Darling, HarmonyOS SDK sont optionnels — les modules de compatibilité buildent des libs auto-contenues qui peuvent se lier à ces backends à runtime via dlopen.

```
# Vendorisation minimale (~50 MB, ~1 min) $ ./scripts/fetch-deps.sh vulkan $ ./scripts/fetch-deps.sh glslang # Vérification $ ./scripts/check-deps.sh [ok] vulkan-headers: present (v1.3.290) [ok] glslang: present (14.2.0) [warn] edk2: missing (required for firmware build) [info] Optional: wine, art, darling, harmony, mesa, iconv
```

4.3 Vendorisation complète (avec EDK2)

Pour un build complet incluant le firmware UEFI, vendorisez EDK2 (~5 GB, téléchargement de 30-60 min selon votre connexion). EDK2 a des submodules qui sont automatiquement récupérés par le script.

```
# Vendorisation EDK2 seule (~5 GB, 30-60 min) $ ./scripts/fetch-deps.sh edk2 # OU vendorisation tout-en-un (~6.5 GB, 60-90 min) $ ./scripts/fetch-deps.sh all # Progression typique [info] edk2: cloning https://github.com/tianocore/edk2.git (pin=edk2-stable202408) [info] edk2: submodules update --init --depth 1 ... 30-60 minutes plus tard ... [ok] edk2: pinned at edk2-stable202408 (5.2 GB) tree sha256:
```

```
3a7b...e1f9 [info] Total: 9 deps vendored, 6.5 GB on disk
```

4.4 Vérification post-vendorisation

```
$ ./scripts/check-deps.sh [ok] edk2: present (edk2-stable202408) [ok] vulkan-headers: present (v1.3.290) [ok] glslang: present (14.2.0) [ok] wine: present (wine-9.0) [ok] art: present (android-14.0.0_r1) [ok] darling: present (0.1.20240301) [ok] harmony: present (5.0.0-Release) [ok] mesa: present (mesa-24.0.3) [ok] iconv: present (v1.17) [ok] all mandatory dependencies satisfied
```

4.5 Mise à jour d'une dépendance

Pour mettre à jour une dépendance vers une nouvelle version, utilisez le script update-dep.sh qui met à jour le pin dans fetch-deps.sh, re-fetch la dépendance, et génère un diff à committer :

```
$ ./scripts/update-dep.sh edk2 edk2-stable202502 [info] updating pin in fetch-deps.sh [info] re-fetching edk2 [ok] edk2 updated to edk2-stable202502 [diff] fetch-deps.sh: - [edk2]="edk2-stable202408" + [edk2]="edk2-stable202502" [info] commit suggestion: git add scripts/fetch-deps.sh git commit -m "deps: bump edk2 to stable202502"
```

5. Build des 5 couches de compatibilité

Cette section build les 5 modules de compatibilité applicative + l'orchestrateur CoreBridge. Tous produisent des bibliothèques partagées (.so) ou statiques (.a) qui peuvent être chargées par AfriOS à runtime. Le build ne nécessite pas EDK2.

5.1 Build one-shot (recommandé)

Le script scripts/setup.sh automatise le build de toutes les couches de compatibilité en parallèle. C'est la méthode recommandée pour un premier build :

```
$ ./scripts/setup.sh --skip fetch-edk2 Phase: install-workflows [ok] Workflows staged Phase: syntax-check ... 16s ... [ok] 298/329 files passed syntax check Phase: hal-tests [ok] HAL test runner: PASS (42/42) Phase: compat-tests [ok] compat layers built: - libafros-winbridge.so (104 KB) - libafros-androsandbox.so (1.66 MB) - libafros-harmonygate.so (385 KB) - libafros-dxvk.so (434 KB) + 4 Wine DLL aliases - libafros-incompat-engine.so (109 KB, .m excluded) - libafros-corebridge.a (148 KB) [ok] setup complete — every critical phase passed
```

5.2 Build manuel module par module

Pour comprendre chaque build ou pour déboguer un échec, vous pouvez builder chaque module individuellement avec CMake. Tous les modules suivent la même convention :

```
# Pattern générique $ cd AfriOS/AfriOS/OS/afros-<module> $ cmake -B build -S . $ cmake --build build -j$(nproc) # Résultat : build/libafros-<module>.so
```

afros-winbridge (Wine)

```
$ cd AfriOS/AfriOS/OS/afros-winbridge $ cmake -B build -S . && cmake --build build -j4 [100%] Linking C shared library libafros-winbridge.so [100%] Built target afros-winbridge $ ls build/libafros-winbridge.so build/libafros-winbridge.so (104 KB, 158 symboles)
```

afros-androsandbox (Android)

```
$ cd AfriOS/AfriOS/OS/afros-androsandbox $ cmake -B build -S . && cmake --build build -j4 [100%] Linking CXX shared library libafros-androsandbox.so $ ls build/libafros-androsandbox.so build/libafros-androsandbox.so (1.66 MB, 315 symboles)
```

afros-dxvk (DirectX → Vulkan)

```
$ cd AfriOS/AfriOS/OS/afros-dxvk $ cmake -B build -S . && cmake --build build -j4 [100%] Linking CXX shared library libafros-dxvk.so [100%] Built target afros-dxvk [100%] Creating Wine DLL aliases: d3d9.dll.so, d3d11.dll.so, d3d12.dll.so, dxgi.dll.so $ ls build/ libafros-dxvk.so d3d9.dll.so d3d11.dll.so d3d12.dll.so dxgi.dll.so # Les .dll.so sont des symlinks vers libafros-dxvk.so
```

afros-incompat-engine (iOS/macOS)

Note : si gobjc ou clang n'est pas installé, les fichiers .m (UIKit, Foundation, CoreAnimation) sont exclus du build. La lib contient toujours les composants C-only (Mach-O loader, objc runtime C, sandbox, codesign, CoreGraphics).

```
$ cd AfriOS/AfriOS/OS/afros-incompat-engine $ cmake -B build -S . # warning si pas de gobjc $ cmake --build build -j4 [100%] Built target afros-incompat-engine $ ls build/libafros-incompat-engine.so build/libafros-incompat-engine.so (109 KB sans .m, 232 symboles) # Avec gobjc: ~500 KB avec UIKit/Foundation/CoreAnimation frameworks
```

afros-harmonygate (HarmonyOS)

```
$ cd AfriOS/AfriOS/OS/afros-harmonygate $ cmake -B build -S . && cmake --build build -j4 [100%] Built target afros-harmonygate $ ls build/libafros-harmonygate.so build/libafros-harmonygate.so (385 KB, 140 symboles)
```

afros-corebridge-core (orchestrateur)

L'orchestrateur est built en bibliothèque statique (.a) car il est lié directement dans le binaire afros-launch qui constitue le point d'entrée utilisateur.

```
$ cd AfriOS/AfriOS/OS/afros-corebridge-core $ cmake -B build -S . && cmake --build build -j4 [100%] Linking C static library libafros-corebridge.a $ ls build/libafros-corebridge.a build/libafros-corebridge.a (148 KB, 102 symboles)
```

6. Build du noyau (4 architectures)

Le noyau AfriOS est multi-architectures. Chaque build cible une architecture via la variable CMake AFROS_PORT. Vous pouvez builder les 4 architectures en parallèle dans des répertoires de build séparés.

6.1 Vue d'ensemble

Architecture	AFROS_PORT	Cross-compiler	Output
ARM64	arm64	aarch64-linux-gnu-gcc	afros-kernel-arm64.elf
x86_64	x86_64	gcc (natif)	afros-kernel-x86_64.elf
RISC-V 64	riscv	riscv64-linux-gnu-gcc	afros-kernel-riscv.elf
MCU Cortex-M	mcu	arm-none-eabi-gcc	afros-kernel-mcu.elf
Host mock	host-mock	gcc (natif)	afros-kernel-host-mock (tests)

6.2 Build ARM64 (Raspberry Pi 4, Pine64, Ampere Altra)

```
$ cd AfriOS/AfriOS/OS $ cmake -B build-arm64 -S . \ -DAFROS_PORT=arm64 \
-DAFROS_BUILD_KERNEL=ON \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-arm64.cmake $ cmake --build
build-arm64 -j$(nproc) $ ls build-arm64/afros-kernel-arm64.elf
build-arm64/afros-kernel-arm64.elf (~1.5 MB)
```

6.3 Build x86_64 (desktops, serveurs)

```
$ cd AfriOS/AfriOS/OS $ cmake -B build-x86_64 -S . \ -DAFROS_PORT=x86_64 \
-DAFROS_BUILD_KERNEL=ON \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-x86_64.cmake $ cmake --build
build-x86_64 -j$(nproc) $ ls build-x86_64/afros-kernel-x86_64.elf
build-x86_64/afros-kernel-x86_64.elf (~1.8 MB)
```

6.4 Build RISC-V 64 (SiFive, QEMU virt)

```
$ cd AfriOS/AfriOS/OS $ cmake -B build-riscv -S . \ -DAFROS_PORT=riscv \
-DAFROS_BUILD_KERNEL=ON \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-riscv.cmake $ cmake --build
build-riscv -j$(nproc) $ ls build-riscv/afros-kernel-riscv.elf
build-riscv/afros-kernel-riscv.elf (~1.4 MB)
```

6.5 Build MCU Cortex-M (STM32, nRF52, RP2040)

Le build MCU active automatiquement AFROS_FREESTANDING=ON et utilise le linker script dédié linker-mcu.ld (memory layout : FLASH 0x08000000 512K, RAM 0x20000000 128K).

```
$ cd AfriOS/AfriOS/OS $ cmake -B build-mcu -S . \ -DAFROS_PORT=mcu \
-DAFROS_BUILD_KERNEL=ON \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-mcu.cmake $ cmake --build
```

```
build-mcu -j$(nproc) $ ls build-mcu/afros-kernel-mcu.elf
build-mcu/afros-kernel-mcu.elf (~280 KB, binaire firmware)
```

6.6 Build host-mock (pour tests sans QEMU)

Le port host-mock est spécial : il permet d'exécuter les tests HAL sur un Linux host sans bare-metal. Il n'est pas destiné à la production mais est crucial pour la CI/CD.

```
$ cd AfriOS/AfriOS/OS $ cmake -B build-host-mock -S . \ -DAFROS_PORT=host-mock \
-DAFROS_BUILD_TESTS=ON $ cmake --build build-host-mock -j$(nproc) $ ctest
--test-dir build-host-mock --output-on-failure Test project build-host-mock Start
1: hal_test_runner 1/1 Test #1: hal_test_runner ......... Passed 0.5s 100% tests
passed, 0 tests failed out of 1
```

7. Build du firmware UEFI

Le build firmware UEFI nécessite EDK2 vendorisé (5 GB). Il produit des fichiers .fd (flash descriptor) bootables sur QEMU ou hardware réel. Le firmware supporte 3 architectures : X64, AARCH64, RISC-V.

7.1 Pré-requis EDK2

Vérifiez qu'EDK2 est bien vendorisé avant de continuer :

```
$ ls external/edk2/edksetup.sh external/edk2/edksetup.sh # doit exister $ du -sh external/edk2 5.2G external/edk2 # doit faire ~5 GB # Si manquant, vendoriser $ ./scripts/fetch-deps.sh edk2
```

7.2 Configuration de l'environnement EDK2

EDK2 utilise son propre système de build (basé sur build et des fichiers .dsc). Vous devez sourcer edksetup.sh avant d'invoquer build.

```
$ cd AfriOS/FirmwareHybride/edk2 $ source edksetup.sh $ export WORKSPACE=$(pwd) $ export PACKAGES_PATH=$WORKSPACE:$WORKSPACE/HybridFirmwarePlatformPkg # Vérification $ which build /home/user/AfriOS/external/edk2/BaseTools/BinWrappers/LinuxLike/build
```

7.3 Build firmware X64 (QEMU, serveurs x86)

```
$ cd AfriOS/FirmwareHybride/edk2 $ source edksetup.sh $ build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a X64 -t GCC5 -b RELEASE # Output $ ls Build/HybridFirmwarePlatformPkg/RELEASE_GCC5/FV/ HYBRIDFIRMWAREPLATFORMPKG.FD (~4 MB flash image) HYBRIDFIRMWAREPLATFORMPKG.Fv # firmware volume
```

7.4 Build firmware AARCH64 (Rpi 4, Pine64)

```
$ cd AfriOS/FirmwareHybride/edk2 $ source edksetup.sh $ build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a AARCH64 -t GCC5 -b RELEASE $ ls Build/HybridFirmwarePlatformPkg/RELEASE_GCC5/FV/ HYBRIDFIRMWAREPLATFORMPKG.FD (~5 MB flash image)
```

7.5 Build pour une carte spécifique

Le firmware supporte des BoardConfigs prédéfinis pour cartes réelles. Passez la variable BOARD_CONFIG au build pour activer les overrides (FDT base address, etc.).

```
# Raspberry Pi 4 $ build -p HybridFirmwarePlatformPkg.dsc -a AARCH64 -t GCC5 \ -D BOARD_CONFIG=Rpi4 # Pine64 $ build -p HybridFirmwarePlatformPkg.dsc -a AARCH64 -t GCC5 \ -D BOARD_CONFIG=Pine64 # QEMU virt ARM64 $ build -p HybridFirmwarePlatformPkg.dsc -a AARCH64 -t GCC5 \ -D BOARD_CONFIG=QemuVirt # QEMU virt RISC-V $ build -p HybridFirmwarePlatformPkg.dsc -a RISCV64 -t GCC5 \ -D BOARD_CONFIG=QemuVirtRiscv
```

7.6 Test firmware sur QEMU

Une fois le firmware buildé, vous pouvez le booter dans QEMU pour valider qu'il démarre correctement :

```
# QEMU ARM64 virt avec firmware AfriOS $ qemu-system-aarch64 \ -machine virt \ -cpu cortex-a72 \ -m 2G \ -bios Build/HybridFirmwarePlatformPkg/RELEASE_GCC5/FV/HYBRIDFIRMWAREPLATFORMPKG.FD \ -nographic # QEMU X64 (OVMF-style) $ qemu-system-x86_64 \ -m 2G \ -bios Build/HybridFirmwarePlatformPkg/RELEASE_GCC5/FV/HYBRIDFIRMWAREPLATFORMPKG.FD \ -nographic # Sortie attendue (console UEFI) UEFI firmware AfriOS (HybridFirmwarePlatformPkg) v1.0 Platform: QEMU virt ARM64 (BOARD_CONFIG=QemuVirt) Press ESC for boot menu... Starting AfriOS kernel...
```

8. Tests et validation

Une fois les builds terminés, vous pouvez lancer la suite de tests pour valider que tout fonctionne correctement. AfriOS dispose de 4 niveaux de tests.

8.1 Niveau 1 — Tests HAL (sans QEMU)

Tests de la Hardware Abstraction Layer via le port host-mock. 42 tests couvrent l'init/deinit, les opérations CPU, mémoire, interruptions, timer, console, stockage, et devices. S'exécutent en 0.5 s sur n'importe quel Linux host.

```
$ ./scripts/run-hal-tests.sh [HAL] Console initialized [HAL] CPU initialized ... [PASS] hal_init_cycle [PASS] cpu_get_info_core0 [PASS] memory_alloc / memory_free / memory_map_unmap ... Summary: PASS=42, FAIL=0, SKIP=2 [ok] all HAL tests passed
```

8.2 Niveau 2 — Tests syntaxiques CI

Validation syntaxique de tous les fichiers source (C, C++, ASM) avec gcc -fsyntax-only -Wall. 337 fichiers vérifiés en 16 s en parallèle (8 workers).

```
$ python3 scripts/ci-syntax-check.py ... 300/337 (91%) passed=298 failed=0 skipped=22 ... 337/337 (100%) passed=298 failed=0 skipped=22 Total files: 337 Passed: 298 Failed: 0 Skipped: 22 (Objective-C .m sans gobjc — normal sur CI)
```

8.3 Niveau 3 — Tests de compatibilité

18 tests couvrent les 5 plateformes de compatibilité : 5 Windows, 4 Android, 4 iOS/macOS, 3 HarmonyOS, 2 Linux baseline. Chaque test compile un binaire cible et vérifie son exécution via le runtime AfriOS approprié.

```
$ ./tests/run-compat-tests.sh --platform linux [build] Building tests/linux/hello-elf... [ok] hello-elf built (16 KB ELF) [run] hello-elf ... [ok] stdout match: 'Hello, AfriOS!' [ok] exit code: 0 [score] 100/100 [build] Building tests/linux/fork-exec... [run] fork-exec ... [ok] stdout match: 'child' [ok] exit code: 0 [score] 100/100 Summary: 2/2 tests passed (200/200 score)
```

8.4 Niveau 4 — Boot test sur QEMU

Test ultime : booter le noyau AfriOS cross-compilé sur QEMU virt. Valide que le binaire est bootable et que le kernel_main s'exécute.

```
# ARM64 sur QEMU virt $ qemu-system-aarch64 -machine virt -cpu cortex-a72 -m 2G \ -kernel build-arm64/afros-kernel-arm64.elf \ -nographic -no-reboot # Sortie attendue --- Bienvenue dans AfriOS Core (ARM64 v8.5) --- [KERNEL] Démarrage du système de démarrage... [HAL] Console initialized [HAL] CPU initialized (4 cœurs détectés) [HAL] Memory initialized [HAL] Interrupts initialized [HAL] Timer initialized [HAL] Storage initialized [POWER] Source Batterie/AC. Mode économie d'énergie. [KERNEL] Noyau opérationnel. Lancement du planificateur... [KERNEL] Arrêt du système ou Idle...
```

8.5 Script one-shot de validation

```
$ ./scripts/run-all-tests.sh [suite 1/6] syntax-check ..........................
PASS [suite 2/6] hal-tests ........................... PASS (42/42) [suite 3/6]
compat-tests (linux) ............... PASS (2/2) [suite 4/6] mcu link test
....................  SKIP (no arm-none-eabi) [suite 5/6] riscv link test
....................  SKIP (no riscv64-gcc) [suite 6/6] cmake configure
.....................  PASS  Report:  tests/results/full-report-20260903-143000.md
Total: 4 PASS / 2 SKIP / 0 FAIL
```

9. Dépannage

Cette section liste les erreurs les plus courantes rencontrées pendant le build et leurs solutions.

9.1 Erreurs CMake

"No cmake_minimum_required command is present"

Cause : un fichier CMakeLists.txt manque la directive cmake_minimum_required en tête. Solution : vérifier que le patch a été appliqué (tous les CMakeLists.txt doivent commencer par cmake_minimum_required(VERSION 3.16)).

```
# Vérifier $ head -2 AfriOS/AfriOS/OS/afros-*/CMakeLists.txt # Si manquant, ajouter manuellement $ sed -i '1i cmake_minimum_required(VERSION 3.16)\nproject(myproj LANGUAGES C CXX)' \ AfriOS/AfriOS/OS/afros-winbridge/CMakeLists.txt
```

"Could NOT find Threads (missing: Threads_FOUND)"

Cause : find_package(Threads REQUIRED) est appelé avant la déclaration de la target. Solution : déplacer find_package(Threads) AVANT target_link_libraries(... Threads::Threads).

9.2 Erreurs cross-compilation

"aarch64-linux-gnu-gcc: command not found"

Cause : le cross-compiler ARM64 n'est pas installé. Solution :

```
$ sudo apt-get install gcc-aarch64-linux-gnu $ aarch64-linux-gnu-gcc --version # vérification
```

"cc1obj: No such file or directory" (afros-incompat-engine)

Cause : le compilateur Objective-C n'est pas installé. Solution 1 (recommandée) : installer gobjc ou clang. Solution 2 (dégradée) : le CMakeLists a un fallback qui exclut les fichiers .m du build — vous obtiendrez une lib sans UIKit/Foundation/CoreAnimation.

```
# Solution 1 (complète) $ sudo apt-get install gobjc gobjc-14 clang libobjc-10-dev # Solution 2 (dégradée — lib sans frameworks .m) # Le fallback se déclenche automatiquement si gobjc absent
```

9.3 Erreurs EDK2

"build: command not found"

Cause : edksetup.sh n'a pas été sourcé. Solution :

```
$ cd AfriOS/FirmwareHybride/edk2 $ source edksetup.sh $ which build # doit maintenant pointer vers BaseTools/BinWrappers
```

"HybridFirmwarePlatformPkg.dsc not found"

Cause : la variable PACKAGES_PATH n'inclut pas le dossier du package. Solution :

```
$ export WORKSPACE=$(pwd) $ export PACKAGES_PATH=$WORKSPACE:$WORKSPACE/HybridFirmwarePlatformPkg $ source edksetup.sh
```

"cannot find BaseTools"

Cause : les submodules EDK2 n'ont pas été récupérés. Solution :

```
$ cd external/edk2 $ git submodule update --init --depth 1 $ make -C BaseTools/Source/C $ source edksetup.sh
```

9.4 Erreurs tests

"afros-launch: command not found"

Cause : le binaire afros-launch (point d'entrée utilisateur de l'orchestrateur) n'est pas encore buildé. Solution provisoire : les tests se mettent en skip gracieux. Solution complète : builder le binaire en linkant libafros-corebridge.a.

"Port-mcu set_frequency returned SUCCESS (expected NOT_SUPPORTED)"

Ce n'est pas une erreur : le port host-mock retourne SUCCESS pour toutes les opérations, y compris celles qui devraient échouer en bare-metal. Le runner a des guards AFROS_TEST_HOST_MOCK pour gérer ces cas. Les 2 SKIP du rapport HAL sont normaux.

10. Check-list finale du build complet

Cette check-list résume toutes les étapes du build OS complet. Cochez chaque case au fur et à mesure. À la fin, vous aurez un build AfriOS complet : firmware UEFI + noyau pour 4 architectures + 5 couches de compatibilité + tests validés.

10.1 Pré-requis

- [ ] gcc/g++ 14+ installé
- [ ] cmake 3.16+ installé
- [ ] python3 3.10+ installé
- [ ] aarch64-linux-gnu-gcc installé
- [ ] riscv64-linux-gnu-gcc installé
- [ ] arm-none-eabi-gcc installé
- [ ] x86_64-w64-mingw32-gcc installé (optionnel)
- [ ] gobjc ou clang installé (optionnel)
- [ ] qemu-system-{aarch64,x86,riscv} installé
- [ ] ./scripts/setup.sh --check-deps passe

10.2 Acquisition

- [ ] Dépôt cloné (git clone ...)
- [ ] ./scripts/fetch-deps.sh vulkan OK
- [ ] ./scripts/fetch-deps.sh glslang OK
- [ ] ./scripts/fetch-deps.sh edk2 OK (~5 GB)
- [ ] ./scripts/check-deps.sh tout OK

10.3 Build couches compatibilité

- [ ] libafros-winbridge.so buildé
- [ ] libafros-androsandbox.so buildé
- [ ] libafros-harmonygate.so buildé
- [ ] libafros-dxvk.so buildé + 4 symlinks DLL
- [ ] libafros-incompat-engine.so buildé (.m ou non)
- [ ] libafros-corebridge.a buildé

10.4 Build noyau

- [ ] afros-kernel-arm64.elf buildé
- [ ] afros-kernel-x86_64.elf buildé
- [ ] afros-kernel-riscv.elf buildé

- [ ] afros-kernel-mcu.elf buildé

10.5 Build firmware

- [ ] HYBRIDFIRMWAREPLATFORMPKG.FD X64 buildé
- [ ] HYBRIDFIRMWAREPLATFORMPKG.FD AARCH64 buildé
- [ ] HYBRIDFIRMWAREPLATFORMPKG.FD AARCH64 Rpi4 buildé (optionnel)
- [ ] HYBRIDFIRMWAREPLATFORMPKG.FD AARCH64 Pine64 buildé (optionnel)
- [ ] HYBRIDFIRMWAREPLATFORMPKG.FD RISCV64 buildé (optionnel)

10.6 Validation

- [ ] ci-syntax-check.py : 0 erreur
- [ ] run-hal-tests.sh : 42 PASS
- [ ] run-compat-tests.sh : Linux baseline OK
- [ ] Boot QEMU ARM64 : kernel_main s'exécute
- [ ] Boot QEMU X64 : UEFI firmware démarre
- [ ] run-all-tests.sh : génère le rapport

10.7 Artifacts finaux attendus

```
build/ +-- compat/ | +-- afros-winbridge/libafros-winbridge.so (104 KB) | +-- afros-androsandbox/libafros-androsandbox.so (1.66 MB) | +-- afros-harmonygate/libafros-harmonygate.so (385 KB) | +-- afros-dxvk/libafros-dxvk.so (434 KB) | +-- afros-dxvk/d3d{9,11,12}.dll.so + dxgi.dll.so (symlinks) | +-- afros-incompat-engine/libafros-incompat-engine.so (109 KB+) | +-- afros-corebridge-core/libafros-corebridge.a (148 KB) +-- hal-tests/hal_test_runner +-- tests/hello-elf +-- tests/fork-exec +-- arm64/afros-kernel-arm64.elf +-- x86_64/afros-kernel-x86_64.elf +-- riscv/afros-kernel-riscv.elf +-- mcu/afros-kernel-mcu.elf +-- firmware/ | +-- X64/HYBRIDFIRMWAREPLATFORMPKG.FD | +-- AARCH64/HYBRIDFIRMWAREPLATFORMPKG.FD | +-- RISCV64/HYBRIDFIRMWAREPLATFORMPKG.FD (optionnel) +-- reports/build-report-latest.md + .json
```

11. Annexe A — Build sur Windows via WSL2

Le build natif Windows n'est pas supporté par AfriOS pour plusieurs raisons structurelles : les cross-compilers (aarch64-linux-gnu, riscv64-linux-gnu) sont des binaires Linux ; EDK2 utilise source edksetup.sh (bash) ; le module afros-winbridge construit Wine, ce qui n'a pas de sens sur Windows natif (Wine simule Windows sur Linux) ; les tests HAL host-mock utilisent POSIX (clock_gettime, mmap, pthread) ; les scripts d'automatisation (fetch-deps.sh, setup.sh, run-all-tests.sh) sont en bash.

La solution recommandée et officiellement supportée est d'utiliser WSL2 (Windows Subsystem for Linux 2) avec Ubuntu 24.04. WSL2 fournit un vrai kernel Linux dans Windows avec des performances quasi-natives (overhead ~5%), un accès direct au système de fichiers Windows via /mnt/c/, et un accès aux fichiers Linux depuis Windows via \\wsl$\Ubuntu-24.04\home\<user>\.

11.1 Pourquoi pas de build natif Windows ?

Le tableau ci-dessous résume les obstacles techniques d'un build natif Windows, et comment WSL2 les résout.

Composant	Problème Windows natif	Solution WSL2
gcc-aarch64-linux-gnu	Binaire Linux uniquement	Cross-compiler Ubuntu dans WSL
gcc-riscv64-linux-gnu	Idem	Idem
EDK2 build	source edksetup.sh (bash)	Bash natif dans WSL
afros-winbridge (Wine)	Wine = simuler Windows sur Linux	Construit dans WSL
Scripts (.sh)	Pas de bash natif	Bash WSL
Code noyau (POSIX)	<sys/types.h>, mmap, pthread	Kernel Linux WSL
Tests HAL host-mock	clock_gettime, select, mmap	API POSIX dans WSL
QEMU boot tests	QEMU Windows existe mais pas KVM	QEMU WSL + KVM si WSLg

11.2 Étape 1 — Installer WSL2 + Ubuntu 24.04

Ouvrez PowerShell en administrateur et lancez :

```
# PowerShell (admin) wsl --install -d Ubuntu-24.04 # Redémarrer Windows si demandé
# Puis lancer Ubuntu wsl # Premier lancement : définir un user Linux + mot de passe
# (ces credentials sont INDEPENDANTS de Windows)
```

Vérifiez la version WSL :

```
# Dans WSL $ uname -a Linux <hostname> 6.6.x-microsoft-standard-WSL2 #1 SMP ... $
cat /etc/os-release | grep PRETTY PRETTY_NAME="Ubuntu 24.04 LTS"
```

11.3 Étape 2 — Installer les pré-requis (une fois)

Dans WSL Ubuntu, installez tous les outils en une seule commande :

```
# Dans WSL Ubuntu sudo apt-get update sudo apt-get install -y \ build-essential gcc g++ make cmake ninja-build \ python3 python3-pip git curl wget unzip pkg-config libssl-dev \ gcc-aarch64-linux-gnu gcc-riscv64-linux-gnu gcc-arm-none-eabi \ x86_64-w64-mingw32-gcc gobjc gobjc-14 clang libobjc-10-dev \ qemu-system-arm qemu-system-x86 qemu-system-misc \ qemu-user-static # Vérifier les versions gcc --version # 14.x cmake --version # 3.28+ aarch64-linux-gnu-gcc --version # 14.x riscv64-linux-gnu-gcc --version # 14.x arm-none-eabi-gcc --version # 13.x qemu-system-aarch64 --version # 8.x
```

11.4 Étape 3 — Cloner AfriOS dans WSL

Clonez le dépôt dans /home/<user>/ (filesystem Linux natif — beaucoup plus rapide que /mnt/c/ pour les builds).

```
# Dans WSL cd ~ git clone https://github.com/ErdisKodjo/System.git AfriOS cd AfriOS ls -la
```

11.5 Étape 4 — Vendorisation + build complet

Une fois le dépôt cloné, suivez exactement les sections 4 à 7 de ce guide — les commandes sont identiques sous WSL car c'est un vrai Linux.

```
# Vendoriser les dépendances (~60 min pour EDK2) ./scripts/fetch-deps.sh all # Build + tests tout-en-un (~70 min) ./scripts/setup.sh --all # Build firmware séparé (nécessite EDK2) cd AfriOS/FirmwareHybride/edk2 source edksetup.sh build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a AARCH64 -t GCC5 -b RELEASE build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a X64 -t GCC5 -b RELEASE
```

11.6 Étape 5 — Accéder aux artifacts depuis Windows

WSL expose automatiquement vos fichiers Linux dans Windows via le chemin \\wsl$\. Pour ouvrir les PDFs générés depuis Windows, vous avez deux options :

```
# Option A : copier les artifacts vers Windows cp AfriOS/AfriOS-0*.pdf /mnt/c/Users/$USER/Desktop/ cp build/reports/build-report-latest.md /mnt/c/Users/$USER/Desktop/ # Option B : ouvrir l'explorateur Windows à la position actuelle explorer.exe . # Option C : ouvrir un PDF directement powershell.exe start AfriOS-04-Guide-Build-Complet.pdf # Option D : accéder depuis Windows à un path WSL # Dans File Explorer ou n'importe quel Save dialog: # \\wsl$\Ubuntu-24.04\home\<user>\AfriOS\
```

11.7 Étape 6 — Test QEMU (avec KVM si WSLg activé)

Pour le boot test du noyau ARM64 cross-compilé sur QEMU, l'accélération KVM est disponible dans WSL2 sur les machines compatibles. Sans KVM, QEMU fonctionne en mode TCG (emulation logicielle — plus lent mais fonctionnel).

```
# Vérifier KVM dans WSL ls /dev/kvm /dev/kvm # si présent, KVM disponible # Boot test ARM64 sur QEMU virt qemu-system-aarch64 \ -machine virt -cpu cortex-a72 -m 2G \ -kernel build-arm64/afros-kernel-arm64.elf \ -nographic -no-reboot \ $( [ -e /dev/kvm ] && echo '-enable-kvm' ) # Sortie attendue --- Bienvenue dans AfriOS Core (ARM64 v8.5) --- [KERNEL] Démarrage du système de démarrage... [HAL] Console
```

```
initialized ...
```

11.8 Optimisations WSL2

Mémoire et CPU

Par défaut WSL2 limite à 50 % de votre RAM et 4 cœurs. Pour les builds EDK2 (gourmands), augmentez ces limites en créant C:\Users\<user>\.wslconfig :

```
# C:\Users\<user>\.wslconfig (côté Windows) [wsl2] memory=8GB processors=8 swap=4GB # Relancer WSL pour appliquer wsl --shutdown wsl
```

VS Code + extension WSL

Pour éditer le code AfriOS confortablement :

```
# Installer VS Code côté Windows (https://code.visualstudio.com) # Puis dans VS Code : installer l'extension 'WSL' (ms-vscode-remote.remote-wsl) # Depuis WSL, dans le dossier AfriOS : code . # VS Code s'ouvre en mode 'remote-WSL' — toutes les commandes # (build, debug, syntax check) tournent dans WSL, l'UI dans Windows
```

Performance filesystem

Les builds sont 3 à 5x plus rapides si le dépôt est sur le filesystem Linux (/home/<user>/) plutôt que sur Windows (/mnt/c/). Évitez de cloner le dépôt dans /mnt/c/Users/.../ sauf si vous devez éditer depuis un IDE Windows sans extension WSL.

11.9 Limites connues de WSL2 pour AfriOS

- Pas de serial hardware : WSL2 n'expose pas /dev/ttyUSB*. Pour flasher un firmware sur hardware réel (Rpi4, STM32), utilisez l'outil Windows natif (e.g., Raspberry Pi Imager, STM32CubeProgrammer) avec le .FD/.elf copié depuis WSL.
- Pas de GPU direct : pas d'accès au pilote Vulkan Windows depuis WSL. Les tests DXVK peuvent être compilés mais pas exécutés (pas de ICD Vulkan dans WSL2). Pour tester l'exécution réelle DXVK, il faut Linux natif ou WSLg avec vulkan-sdk installé.
- Pas de boot sur hardware réel : WSL2 ne permet pas d'écrire directement sur une carte SD ou un module. Utilisez Windows pour le flashing.
- Réseau NAT : WSL2 utilise un réseau NAT. Les tests SoftBus (discovery BLE/mDNS) entre deux instances WSL peuvent nécessiter --networkingMode=mirrored (WSL 2.0+) ou un port forwarding.

11.10 Alternative : Docker Desktop

Si vous ne souhaitez pas installer WSL2, Docker Desktop avec WSL2 backend peut aussi exécuter le build. Limites : pas de KVM (donc pas de QEMU accéléré), pas d'accès direct au filesystem Linux, mais la compilation fonctionne.

```
# Dockerfile minimal pour AfriOS FROM ubuntu:24.04 RUN apt-get update && apt-get install -y \ build-essential cmake python3 git \ gcc-aarch64-linux-gnu gcc-riscv64-linux-gnu gcc-arm-none-eabi # Build + run docker run -it --rm -v ${PWD}:/AfriOS ubuntu:24.04 bash -c \ "cd /AfriOS && ./scripts/setup.sh --skip fetch-edk2"
```

12. Raccourci one-shot

Si vous souhaitez tout faire en une seule commande (après avoir installé les pré-requis), le script setup.sh --all orchestre toutes les phases :

```
# 1. Installer pré-requis (une fois) sudo apt-get install -y build-essential cmake python3 git \ gcc-aarch64-linux-gnu gcc-riscv64-linux-gnu \ gcc-arm-none-eabi x86_64-w64-mingw32-gcc \ gobjc clang qemu-system-{aarch64,x86,riscv} # 2. Cloner git clone https://github.com/ErdisKodjo/System.git AfriOS cd AfriOS # 3. Vendoriser dépendances (~60 min pour EDK2) ./scripts/fetch-deps.sh all # 4. Build + tests tout-en-un (~15 min) ./scripts/setup.sh --all # 5. Build firmware séparé (~30 min, nécessite EDK2) cd AfriOS/FirmwareHybride/edk2 source edksetup.sh build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a AARCH64 -t GCC5 -b RELEASE build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a X64 -t GCC5 -b RELEASE # 6. Vérifier le rapport final cat build/reports/build-report-latest.md
```

11.1 Sortie attendue du setup.sh --all

Phase Result Elapsed Critical ---------- ---------- ---------- ---------- check-deps pass 1s 1 install-workflows pass 0s 1 fetch-edk2 pass 1800s 0 syntax-check pass 16s 1 hal-tests pass 1s 1 compat-tests pass 120s 1 kernel-build-arm64 pass 120s 1 kernel-build-x86_64 pass 60s 1 kernel-build-riscv pass 90s 1 kernel-build-mcu pass 30s 1 firmware-build pass 1800s 0 summary pass 1s - Critical failures: 0 Optional failures: 0 Total elapsed: ~70 minutes [ok] setup complete — every critical phase passed

13. Conclusion

Ce guide vous a conduit pas-à-pas depuis le clone du dépôt jusqu'à un build OS complet d'AfriOS : firmware UEFI multi-architectures, noyau pour 4 architectures, 5 couches de compatibilité applicative, et tests validés. À ce stade, vous disposez d'un environnement de développement AfriOS pleinement fonctionnel.

Pour aller plus loin : (1) consultez le document Nouvelle architecture pour comprendre la vision 6 couches et le flux d'exécution d'une application multi-runtime ; (2) consultez le document Modules pour le détail de chaque sous-système ; (3) consultez le White paper optimisations africain pour les choix techniques spécifiques au contexte africain ; (4) consultez l'API.md de CoreBridge pour développer des applications tierces sur AfriOS.

Pour contribuer au projet : (1) forkez le dépôt ; (2) créez une branche feature/<votre-feature> ; (3) soumettez une PR en suivant le template .github/pull_request_template.md ; (4) pour les changements d'API publique CoreBridge, ouvrez d'abord une RFC dans afros-corebridge-core/rfcs/ selon le processus documenté dans RFC-PROCESS.md.

Le projet AfriOS est open source et accueille les contributions. Les axes prioritaires sont : (a) brancher les stubs cryptographiques sur une vraie HAL crypto ; (b) implémenter le vrai compilateur HLSL → SPIR-V dans DXVK ; (c) valider les tests de compatibilité sur des apps réelles (Notepad++, WhatsApp, etc.) ; (d) documenter les résultats de benchmarks comparatifs AfriOS vs Linux vanilla.